Lesson 3.1 — Anatomy of a Prompt

Lesson 3.1 — Anatomy of a Prompt

> **A production-grade prompt has six parts. Most engineers write prompts with two.** This lesson is about the four parts you are skipping.

By the end of this lesson:

- You will know the six components of a great prompt.
- You will be able to look at any prompt of yours and identify which components are missing.
- You will be able to walk an interviewer through a real prompt and explain every choice.
- You will have a one-line opening that signals seniority on prompts.

---

1. The interviewer test

Imagine your interviewer says: *"Show me the last prompt you wrote, and walk me through the choices in it."*

The bad version of this answer:

> *"Uh, I think I asked it to add a loading state to a component."*

The good version:

> *"Here is the prompt. The verb is 'add' — narrow scope. The deliverable is named — `<LoadingState>` in `components/ui/`. I named two constraints: do not change existing styling, and use the existing `<Spinner>` component. I gave context: the file path of the parent and a paste of the existing JSX. I named one*

failure mode: the loading state must be announced to screen readers via `aria-live`. And I asked for the diff in unified format. Six components, deliberately."

That is the kind of answer that ends interviews early in your favor. **The six components** are the secret sauce.

---

2. The six components

```
┌──────────────────────────────────────────────┐
│ 1. VERB                          │ add / refactor / extract / explain
├──────────────────────────────────────────────┤
│ 2. DELIVERABLE                    │ what gets produced (specific file/component
├──────────────────────────────────────────────┤
│ 3. CONSTRAINTS                              │ what NOT to do
├──────────────────────────────────────────────┤
│ 4. CONTEXT                        │ files, paths, existing patterns referenced
├──────────────────────────────────────────────┤
│ 5. FAILURE MODES                  │ edge cases the model would otherwise miss
├──────────────────────────────────────────────┤
│ 6. OUTPUT FORMAT                  │ diff / single file / explanation / shell com
└──────────────────────────────────────────────┘
```

Most engineers write prompts that are basically just rows 1 and 2. The other four are where the leverage is.

---

3. Component 1 — The verb

Every prompt should start with a single, narrow verb.

Bad verb	Why bad	Better verb
"Make"	Too broad. Implies "design and implement".	"Add" / "Create" / "Generate"

Bad verb	Why bad	Better verb
“Fix”	Tells the AI to find a bug, instead of you telling it which one.	“Replace X with Y because Z”
“Improve”	No target metric. Becomes a stylistic rewrite.	“Reduce render count by…”
“Help me with”	The model has to guess your goal.	“Show three approaches to…”

The verb library

If you want…	Use this verb
Code that does not exist yet	**Create** / **Generate** / **Add**
Code that exists but should change	**Refactor** / **Replace** / **Rewrite**
Code split into smaller code	**Extract**
Understanding	**Explain** / **Walk through**
Choices laid out	**Compare** / **List approaches to**
A bug found	**Diagnose** / **Trace**
A bug fixed	**Patch** (with the bug specified)
Tests added	**Cover** / **Test**
Names changed	**Rename**

The verb sets the scope. **A wrong verb is the most common reason a prompt produces too-large or too-vague output.**

---

4. Component 2 — The deliverable

Be specific about what is produced. The model is not allowed to surprise you.

Bad

> *“Add a loading state.”*

That could be a hook, a component, a CSS class, a wrapper. The model picks one and you accept whatever pops out.

Good

> *"Add a `<LoadingState>` component to `components/ui/loading-state.tsx`. Default export. Props: `label` (optional, defaults to 'Loading…'), `size` ('sm' | 'md' | 'lg', defaults to 'md'). Render a centered spinner with the label below it."*

Now there is no ambiguity.

What to specify

- **Where it lives.** File path or directory.
- **What it is.** Component / function / hook / type / config / test.
- **Its name.** Pin the name to avoid the model inventing a verbose one.
- **Its API surface.** Props, parameters, return type.
- **Its rendering or behavior.** One sentence is enough.

The deliverable section is where most prompts collapse. Spend extra time on it. **The clearer the deliverable, the smaller the diff and the easier the review.**

---

5. Component 3 — Constraints (the "do not" list)

This is the component most engineers skip. It is the one with the highest payoff.

Examples that work

- *"Do not introduce new dependencies."*
- *"Do not change the existing styling."*
- *"Do not modify any file outside `components/ui/`."*
- *"Do not introduce a Context — pass props."*
- *"Do not generate tests in this turn — I will ask separately."*
- *"Do not refactor unrelated code."*
- *"Do not change the public API of `useSnookerScores`."*
- *"Do not add comments unless I ask."*

Why this matters in interviews

Interviewers reading your prompts can tell who has been burned by AI sprawl. **Constraint clauses are the scar tissue of experienced AI users.** Show your scars.

If they ask why you write so many "do not" clauses, you say:

> *"Each one of those clauses is a real failure mode I have hit. The model is helpful by default — sometimes too helpful. If I do not pin down what stays the same, it will rewrite things I did not ask about. Constraints make diffs reviewable."*

---

6. Component 4 — Context

The model needs to know what is already there. Be explicit.

Three kinds of context

6.1. Path context

> *"The parent component is at `components/predictions/PredictionMatrix.tsx`. Use the existing styles from `lib/constants.ts` (`th`, `td`, `tabStyle`)."*

This tells the model where to look for patterns to match. Modern agentic tools (G4 generation from Lesson 1.1) can read those files automatically — but stating them in the prompt removes ambiguity.

6.2. Pattern context

> *"Match the structure of `PlayerCard.tsx` — same prop shape, same Tailwind classes, same conditional logic for badges."*

You are pointing to a sibling that should be the template. The model imitates rather than invents.

6.3. Snippet context

> *"Here is the existing JSX I want you to wrap…"*

For chat-style prompts, paste the actual snippet. For agentic prompts, point to the file and line range.

What NOT to include

- **Your whole codebase.** The model gets confused by noise.
- **Irrelevant history.** "Previously we tried X…" → only mention if the failed attempt is informative.
- **Personal context.** "My PM is annoying about this…" → the model does not need this.

The right context is the smallest set of files / patterns / snippets that pin down what good output looks like.

---

7. Component 5 — Failure modes

Tell the model what it would otherwise get wrong.

Examples

- *"The empty array case must render an explicit empty state — not just nothing."*
- *"Negative numbers should not be possible — clamp to zero in the helper."*
- *"This API call may return 404 if the league has not started — handle that explicitly."*
- *"The user might have unfinished picks. Treat unfinished as a third state, not as wrong."*
- *"Avoid Date.now() — use the `clock` parameter that is passed in for testability."*

These are statements about the **shape of the world** the model would otherwise default-to-not-knowing. They prevent 80% of "the AI was wrong" frustrations.

Where do failure modes come from?

Three sources:

1. **Bugs you have hit before** in this codebase. Add a clause to prevent the AI from re-introducing them.
2. **Data shape edge cases** — empty, null, very large, locale-specific.
3. **Domain knowledge the AI cannot infer**. "In our domain, X means Y, not the standard meaning."

---

8. Component 6 — Output format

Tell the model how to present its output.

Common formats and when to use each

Format	When
Unified diff	Refactor / patch / change to existing file
Single complete file	Brand-new file
List of file paths + contents	Multi-file scaffold
Explanation prose, no code	"Compare three approaches"
Step-by-step plan	Before any code is written, for big features
Shell commands	Setup, install, scaffold tasks
JSON block	Structured data extraction

Why this matters

Without a format, the model picks one. Half the time it is wrong for your workflow. Specifying the format makes the output **immediately usable** — paste-friendly, diff-applicable, or grep-able.

A senior tell: *"Return as a unified diff with no surrounding prose."* That tells the model: I am a professional reviewer; do not pad your answer.

---

9. A worked example — bad to good

v0 — The naive prompt

> *"add a loading state to my button"*

What is wrong: missing 4 of 6 components.

v1 — Add the deliverable

> *"Add a loading state to `<PrimaryButton>` in `components/ui/PrimaryButton.tsx`. When `isLoading` is true, show a spinner inside the button and disable it."*

Better. Has verb, deliverable, some context.

v2 — Add constraints

> *"Add a loading state to `<PrimaryButton>` in `components/ui/PrimaryButton.tsx`. When `isLoading` is true, show a spinner inside the button and disable it. Do not change the existing prop API. Do not add a new dependency for the spinner — use `<Spinner>` from the same folder."*

Now diff size is bounded.

v3 — Add failure modes

> *"Add a loading state to `<PrimaryButton>` in `components/ui/PrimaryButton.tsx`. When `isLoading` is true, show a spinner inside the button and disable it. Do not change the existing prop API. Do not add a new dependency for the spinner — use `<Spinner>` from the same folder. The button must remain keyboard-focusable while loading (focus ring visible) but click handlers should be no-ops. Announce the state change with `aria-busy`."*

Now accessibility is locked in.

v4 — Add output format

> *"Add a loading state to `<PrimaryButton>` in `components/ui/PrimaryButton.tsx`. When `isLoading` is true, show a spinner inside the button and disable it. Do not change the existing prop API. Do not add a new dependency for the spinner — use `<Spinner>` from the same folder. The button must remain keyboard-focusable while loading (focus ring visible) but click handlers should be no-ops. Announce the state change with `aria-busy`. Return a unified diff. No prose."*

That is a senior-grade prompt. **Walk an interviewer through this and they will know you are a craftsman.**

---

10. The opening line that signals seniority

In any conversation about how you prompt, drop this:

> *"I think of a prompt the way I think of a function signature. Verb, deliverable, constraints, context. If a prompt is missing any of those, my output will be."*

That sentence does three things:

1. Compares prompting to a familiar engineering primitive.
2. Lists the components.
3. Implies that you treat prompt quality as a measurable craft.

Use it once per interview. Just once. Save it for when the topic of "how do you prompt?" comes up naturally.

---

11. The "system prompt" question

Some interviewers ask: *"Do you use system prompts? Custom instructions?"* Have an answer.

> *"Yes. I keep a project-level system prompt that pins my codebase conventions — TypeScript strict, no default exports for components except for routes, prefer hooks over HOCs, our internal `cn()` utility for classnames. That replaces about a third of what I would otherwise type into every individual prompt. Treat the system prompt like a `.eslintrc` for the AI: it sets defaults so the per-prompt content can focus on the unique deliverable."*

That answer signals you have actually configured your tools, not just used them with defaults. That is a senior signal.

---

12. Practice exercise

Take three prompts you wrote in the last week. For each, score it on the six components:

```
PROMPT 1
  Verb:          [present / missing] _____________________________
  Deliverable:    [precise / vague]  _____________________________
  Constraints:     [N stated]        _____________________________
  Context:       [referenced files]  _____________________________
  Failure modes:  [N stated]         _____________________________
  Output format:  [stated / not]     _____________________________
```

```
PROMPT 2 ...
PROMPT 3 ...
```

Then **rewrite** each one to hit all six components. Compare the original output to what the rewritten prompt would produce. The difference is the lesson. Notice the diff size shrink and the correctness rise.

---

13. Common pitfalls

13.1. Over-specifying

A prompt with twelve constraints, three pages of context, and seven failure modes is overkill. The model gets confused. **Stop at six components, with two or three items per component.**

13.2. Hidden assumptions

> *"Add a button using our standard pattern."*

What is "our standard pattern"? The model has no idea. Either reference the file that defines the pattern or paste a snippet.

13.3. Mixing modes

> *"Refactor this and also add tests and explain how it works and update the docs."*

Four deliverables in one prompt. The model will do a mediocre job of all four. Split into four prompts.

13.4. Prompting before thinking

The fastest way to write a slow prompt is to skip the thinking step. Spend 30 seconds composing a hypothesis ("I think this should be X") before writing the prompt. The hypothesis improves the prompt.

---

14. CHECK YOURSELF

- ☐ Can you list the six components of a prompt from memory?
- ☐ For each component, can you give one good and one bad example?
- ☐ Can you rewrite a vague prompt into a six-component version on the fly?
- ☐ Can you deliver the section-10 senior-signaling line without notes?
- ☐ Have you scored three of your own real prompts using the rubric?

---

15. Where you are now

You have a structural model of what makes a prompt good and a vocabulary for explaining it to interviewers. Move on to **02-the-twelve-patterns.md** — twelve reusable prompt templates you can deploy verbatim.